

Padrões de projeto

**Aplicados a agentes de IA, em estilo
funcional**

O que é um padrão de projeto, de verdade

- Um padrão é uma solução já testada pra um problema que aparece repetido em produção, com nome próprio pra time inteiro falar a mesma língua.
- Exemplos de 4 padrões GoF, todos resolvendo dor real de quem lida com LLM: trocar provedor sem quebrar nada, padronizar resposta bagunçada, monitorar execução.
- Os livros clássicos (Gamma et al.) explicam tudo em classe e herança. Aqui aplicamos o mesmo raciocínio em TypeScript funcional, com função e tipo no lugar de classe.

Factory: trocar de provedor

```
type ILLMProvider = (prompt: string, systemInstruction?: string) => Promise<string>

const geminiProvider: ILLMProvider = async (prompt) => {
  return "resposta do gemini" // chamada real à API do Gemini
}

const groqProvider: ILLMProvider = async (prompt) => {
  return "resposta da groq" // chamada real à API da Groq
}

function createLLMProvider(nome: "gemini" | "groq"): ILLMProvider {
  return nome === "gemini" ? geminiProvider : groqProvider
}
```

A função `createLLMProvider` decide qual função devolver.

Strategy: comportamento em runtime

```
type SkillHandler = (input: string) => Promise<string>

const skills: Record<string, SkillHandler> = {
  revisarCodigo: async (input) => "revisão gerada",
  gerarDocs: async (input) => "documentação gerada",
  traduzir: async (input) => "tradução gerada",
}

async function executarSkill(nome: string, input: string) {
  const handler = skills[nome]
  if (!handler) throw new Error(`Skill "${nome}" não existe`)
  return handler(input)
}
```

O "algoritmo" que muda em runtime é só a função escolhida do mapa `skills`.

Adapter: padronizar resposta bagunçada

```
interface StandardAIResponse {  
  sucesso: boolean  
  conteudoValido: boolean  
  resultado: string  
  tokensUtilizados?: number  
}  
  
function adaptResponse(raw: string, tokens?: number): StandardAIResponse {  
  const conteudoValido = raw.trim().length > 0  
  return { sucesso: conteudoValido, conteudoValido, resultado: raw.trim(), tokensUtilizados: tokens }  
}
```

Cada provedor devolve string de um jeito diferente. `adaptResponse` é uma função pura: entra bagunça, sai contrato fixo.

Observer: monitorar execução sem acoplar

```
type Listener = (evento: string, dados: unknown) => void
const listeners: Listener[] = []

function onAgentEvent(listener: Listener) {
  listeners.push(listener)
}

function emitAgentEvent(evento: string, dados: unknown) {
  listeners.forEach((l) => l(evento, dados))
}

onAgentEvent((evento, dados) => console.log(`[LOG] ${evento}`, dados))
```

Node já resolve isso nativo com `EventEmitter`, mas o raciocínio é o mesmo: quem quiser saber o que o agente está fazendo se inscreve, sem o agente conhecer quem está ouvindo.

Resumo: os mesmos padrões, em função

- Factory, Strategy, Adapter e Observer resolvem os mesmos problemas descritos no GoF. Aqui eles aparecem como função, tipo e módulo.
- Não quis deixar os exemplos em classe por não gostar da verbosidade e achar POO uma porcaria 🤢.
Mas pode usar classe se quiser.

ADR: registrando decisão arquitetural

- ADR (Architecture Decision Record) é um documento curto que registra por que uma decisão técnica foi tomada, além de qual foi tomada.
- Formato de commit message aplicado a arquitetura: contexto, opções consideradas, escolha final, consequência assumida. Cabe numa página.
- Justifica Strategy vs. Factory pra cada decisão de arquitetura, e o ADR dá o template pra isso ficar registrado e rastreável.

Anatomia de um ADR

- **Título:** numerado e descritivo, ex: "0003 - Strategy para troca de provedor LLM".
- **Status:** proposto, aceito ou substituído. Decisão pode ficar obsoleta depois, e o ADR registra isso em vez de sumir do histórico.
- **Contexto:** qual problema forçou a decisão, que restrição pesou (custo, prazo, rate limit).
- **Decisão:** o que foi escolhido, direto ao ponto.
- **Consequências:** o que fica mais fácil, o que fica mais difícil, o que virou dívida técnica assumida de propósito.

ADR: um exemplo real

Título: 0003 - Strategy para troca de provedor LLM

Contexto: rate limit da Gemini nos horários de pico, precisamos trocar pro Groq sem alterar os agentes que já consomem o provider

Decisão: criar ILLMProvider e createLLMProvider(nome), Strategy funcional em vez de acoplar direto ao SDK

Consequências: mais um arquivo de mapeamento, e o acoplamento entre agente e SDK sai baixo de graça

- Fica versionado junto do código (docs/adr/0003-...md), com histórico de commit igual o resto do projeto.

Para estudar mais: padrões e ADR

- GAMMA et al., *Padrões de Projeto*: Factory, Strategy, Adapter e Observer descritos na origem, em 1994.
- refactoring.guru/pt-br/design-patterns: o mesmo catálogo, em português e com código rodável.
- NYGARD, 2011, *Documenting Architecture Decisions*, cognitect.com/blog/2011/11/15: o post que criou o ADR.
- adr.github.io e github.com/joelparkerhenderson/architecture-decision-record: templates prontos.